

UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA

PROYECTO INTEGRADOR 2026

Nodo Sur

Plataforma digital resiliente para una red de emprendimientos y experiencias regionales

Asignatura	EIN-090B · Taller de Administración de Sistemas
Modalidad	Proyecto grupal con verificación y defensa individual
Evaluación	30% de la calificación final de la asignatura
Ciclo	Cinco entregas progresivas · escala de 0 a 100 puntos

Diseñar · implementar · operar · observar · recuperar

1. Propósito del proyecto

El proyecto integrador transforma un problema empresarial en una plataforma operable. El objetivo no es instalar un CMS y mostrar su portada, sino construir una solución que pueda publicarse, administrarse, diagnosticarse, mantenerse y recuperarse ante fallas. Cada equipo actuará como proveedor de infraestructura y deberá justificar sus decisiones mediante requerimientos, pruebas, métricas, logs y procedimientos reproducibles.

PREGUNTA TRANSVERSAL ¿Qué evidencia demuestra que el negocio puede continuar operando cuando un componente falla?

Resultados de aprendizaje

1. RDA1. Describir e integrar servicios de red relacionándolos con disponibilidad, desempeño, rendimiento y seguridad.
2. RDA2. Analizar políticas de administración de datos y servicios según disponibilidad, desempeño, rendimiento y seguridad.
3. RDA3. Detallar y ejecutar requerimientos de respuesta a incidentes coherentes con las políticas de seguridad.

Qué se evalúa

1. Capacidad para traducir necesidades de negocio en requerimientos verificables y decisiones de arquitectura.
2. Implementación integrada de servicios tradicionales y, posteriormente, de componentes contenerizados.
3. Diagnóstico basado en hipótesis, estado de servicios, puertos, métricas y logs.
4. Seguridad por diseño: mínimo privilegio, segmentación, gestión responsable de credenciales y trazabilidad.
5. Continuidad demostrada mediante copias restaurables, medición de RPO/RTO y reversión.
6. Dominio individual: cada integrante debe operar y explicar cualquier componente de la solución.

2. Caso de negocio: Nodo Sur

Nodo Sur es una red regional que agrupa pequeñas marcas, talleres creativos, productores y operadores de experiencias. Actualmente cada participante vende o recibe reservas por redes sociales, formularios aislados y mensajería. La red proyecta lanzar una campaña conjunta que reunirá catálogo, venta, inscripción a talleres, reservas con cupos limitados, contenidos y atención posventa en un único dominio.

La organización no posee un equipo de desarrollo permanente. Necesita una plataforma basada en software libre o de bajo costo, administrable por personal con capacitación básica y capaz de evolucionar sin rehacerse desde cero. La campaña puede multiplicar el tráfico durante períodos breves, por lo que la plataforma debe detectar degradaciones y mantener el servicio aunque falle uno de los nodos web.

La dirección no entrega una arquitectura cerrada. Define resultados empresariales, restricciones y objetivos de servicio. Cada equipo deberá seleccionar un CMS o aplicación equivalente, proponer la suite de servicios necesaria, implementarla progresivamente y defender sus decisiones.

Incidente que origina el encargo

1. En una campaña piloto el formulario dejó de responder durante casi dos horas y nadie pudo determinar la causa.
2. Algunas solicitudes llegaron duplicadas y otras quedaron únicamente en una cuenta personal de correo.
3. El último respaldo disponible tenía varias semanas y nunca se había probado su restauración.
4. Las credenciales se compartían por mensajería y no existía una forma clara de revocar accesos.
5. Las imágenes y archivos se almacenaban sin una política de capacidad, permisos ni retención.
6. No existían métricas, alertas, bitácora de cambios ni responsables definidos para incidentes.

3. Necesidades y objetivos de servicio

Capacidad	Necesidad del negocio
Plataforma	CMS funcional para catálogo, pedidos o reservas y administración de contenidos.
Publicación	Dominio, DNS, HTTP/HTTPS y certificados validados desde cliente y servidor.
Disponibilidad	Dos nodos web y distribución de solicitudes con comprobaciones de salud.
Datos	Base de datos separada y política explícita para archivos, sesiones y persistencia.
Seguridad	Acceso administrativo restringido, firewall, mínimo privilegio y trazabilidad.

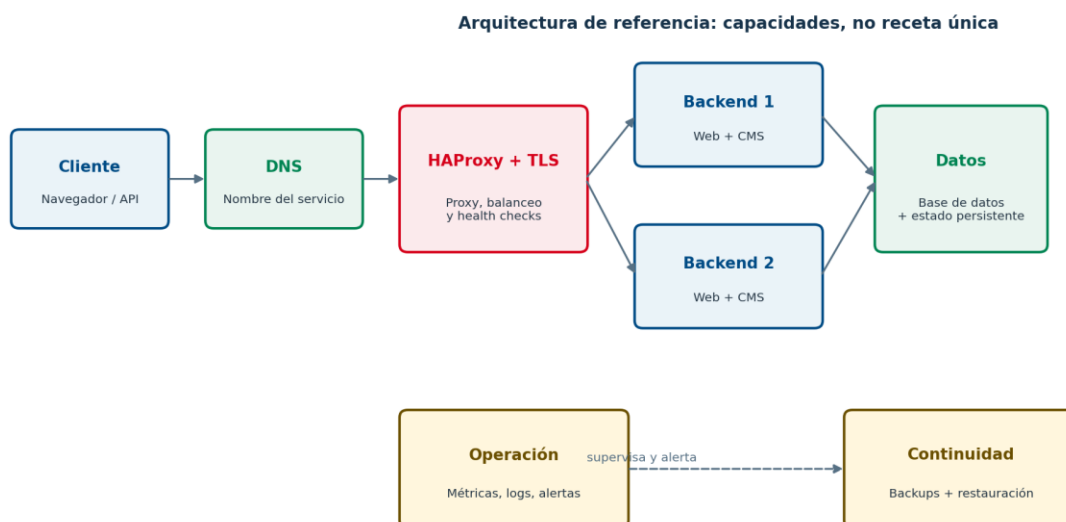
Capacidad	Necesidad del negocio
Operación	Métricas, logs, paneles, alertas, NTP, correo o relay y tareas automatizadas.
Continuidad	Backups de datos, archivos y configuración, más una restauración comprobada.
Incidentes	Procedimiento para detectar, contener, corregir, validar, documentar y revertir.

Objetivos iniciales

Indicador	Meta base	Condición
Disponibilidad mensual	$\geq 99,5\%$	El equipo debe identificar qué componentes siguen siendo puntos únicos de falla.
Detección de falla crítica	≤ 5 min	Alerta verificable, no solo visualización manual.
RTO servicio web	≤ 60 min	Medido en un ejercicio controlado.
RPO general	≤ 24 h	Proponer mejora para datos transaccionales durante campañas.
Retención de respaldos	≥ 30 días	Con política de rotación y al menos una copia separada.
Administración remota	Solo por canales seguros	Accesos nominativos, revocables y con privilegios justificados.

4. Arquitectura mínima y decisiones abiertas

La figura representa capacidades esperadas. No obliga a utilizar una herramienta única ni afirma que la solución completa sea de alta disponibilidad.



Núcleo obligatorio

1. Un CMS o aplicación empresarial aprobada. Referencia: WordPress + WooCommerce; alternativas: PrestaShop, Drupal, Moodle, Nextcloud u otra equivalente justificada.
2. Base de datos en un componente separado y accesible únicamente desde los orígenes autorizados.
3. DNS, servidor web, TLS, firewall y administración remota segura.
4. Dos backends funcionales, HAProxy, health checks y prueba de retiro de un backend defectuoso.
5. Persistencia coherente de base de datos, archivos y cualquier estado necesario para operar con más de un nodo.
6. Monitorización, alertas, logs consultables, correo o relay, sincronización horaria y automatización.
7. Backups de base de datos, archivos y configuración, con restauración controlada y medición de RPO/RTO.
8. Implementación contenerizada reproducible después de haber instalado y diagnosticado los servicios tradicionales.

Decisiones que debe justificar el equipo

1. Selección del CMS, servidor web, motor de base de datos, monitorización y herramienta de backup.
2. Topología, redes, direccionamiento, zonas de confianza y flujos permitidos entre componentes.
3. Qué componentes comparten servidor y cuáles se separan; recursos iniciales y umbrales de crecimiento.
4. Cómo se resolverá el estado compartido, qué se respaldará y qué puede reconstruirse.
5. Métricas, logs, umbrales, responsables y canales de alerta.
6. Puntos únicos de falla aceptados, riesgo residual y evolución recomendada.

5. Restricciones y reglas técnicas

Regla	Aplicación
Software y costo	Privilegiar software libre y recursos compatibles con Proxmox, VMs locales o nube equivalente.
Topología	No resolver toda la plataforma en una sola máquina. Los segmentos e IP se ajustan a la infraestructura disponible.
Orden pedagógico	Implementar y diagnosticar servicios tradicionales antes de migrarlos o reproducirlos con contenedores.
Credenciales	No incluir claves, tokens ni contraseñas en repositorios, capturas, scripts o imágenes.

Regla	Aplicación
Privilegios	No usar chmod 777 como solución. Todo privilegio ampliado debe justificarse y limitarse.
Cambios críticos	Respaldar configuración, validar sintaxis, preferir reload cuando corresponda y documentar reversión.
Pruebas	Validar desde cliente y servidor. Una pantalla “running” o “backup completed” no demuestra cumplimiento.
Contenedores	Usar volúmenes, redes y variables de forma explícita
Alta disponibilidad	Diferenciar balanceo, tolerancia a falla y alta disponibilidad; declarar puntos únicos de falla.
Reproducibilidad	Toda configuración relevante debe poder reconstruirse desde documentación y archivos versionados.

Uso responsable de inteligencia artificial

PERMITIDO CON TRAZABILIDAD Se puede utilizar IA generativa para explorar alternativas, revisar documentación, explicar errores o apoyar scripts. El equipo debe registrar qué utilizó, validar cada salida y defender todas las decisiones. La respuesta de una IA no constituye evidencia técnica.

1. No ingresar credenciales, datos personales ni configuraciones sensibles en servicios externos.
2. Mantener un registro breve: herramienta, propósito, fragmento relevante, validación realizada y cambios aplicados.
3. El código o comando que ningún integrante pueda explicar se considera no defendido.

6. Organización y evaluación

La calificación del proyecto corresponde al 30% de la asignatura. La nota de proyecto se calcula con cinco entregas, equivalentes al 25%, el 5% restante será una note de evaluación de pares, donde cada integrante debe dar una nota a sus compañeros. Cada entrega se califica de 0 a 100 puntos mediante la rúbrica correspondiente.

Entrega	Hito	Ponderación	Producto principal
1	Informe de requerimientos y arquitectura	10%	Documento técnico y defensa breve
2	Plataforma base y CMS funcional	15%	Primera implementación operable
3	Publicación resiliente y seguridad	20%	Dos backends, proxy, contenedores y persistencia
4	Observabilidad y continuidad operacional	25%	Monitorización, alertas, backups y restauración
5	Incidente, auditoría y defensa final	30%	Demostración integral e interrogación individual

Responsabilidad individual

1. El resultado técnico es grupal, pero la evaluación puede ajustarse individualmente según demostración, preguntas y trazabilidad de contribuciones.
2. El docente podrá seleccionar al azar quién ejecuta una prueba, diagnostica una falla o explica un flujo.
3. La ausencia de dominio individual no se compensa con una demostración preparada por otro integrante.
4. Cada entrega debe incluir bitácora de contribuciones y cambios verificables por integrante.

Entrega 1. Informe de requerimientos y arquitectura

PROPÓSITO Convertir el caso de negocio en una propuesta verificable antes de instalar la solución definitiva.

Alcance técnico obligatorio

1. Análisis del problema, actores, activos críticos, flujos de negocio y supuestos.
2. Requerimientos funcionales y no funcionales con criterio de aceptación.
3. Selección del CMS y suite propuesta: licenciamiento, comunidad, operación, límites y alternativas descartadas.
4. Diagrama de despliegue con componentes, redes estimativas, puertos, protocolos, dependencias y flujos de datos.
5. Dimensionamiento inicial, objetivos SLO, RPO/RTO, riesgos, puntos únicos de falla y plan de crecimiento.
6. Modelo de acceso, permisos, secretos, firewall, logs, monitorización, backup y respuesta a incidentes.
7. Plan de implementación por etapas con pruebas, evidencias, reversión y responsables.

Producto que se entrega

1. Informe PDF según plantilla de la asignatura y fuente editable.
2. Diagrama exportado en formato legible.
3. Repositorio inicial con README, estructura, decisiones y bitácora.

Evidencias mínimas

- ☐ Matriz requisito → componente → prueba → evidencia.
- ☐ Diagrama legible y tabla de flujos/puertos.
- ☐ Registro de decisiones de arquitectura con al menos dos alternativas comparadas.
- ☐ Inventario preliminar de VMs, CPU, RAM, disco, redes y dependencias.
- ☐ Riesgos priorizados y tratamiento propuesto.

Demostración y verificación

1. Exponer la propuesta en un máximo de 10 minutos sin leer el informe.
2. Responder preguntas sobre costos, escalamiento, seguridad, estado compartido y recuperación.
3. Modificar oralmente una decisión cuando el docente cambie un supuesto del caso.

Criterios de aceptación

1. La arquitectura satisface las capacidades mínimas sin anticipar tecnologías no justificadas.
2. Cada requerimiento importante posee una forma concreta de validación.
3. Se distinguen hechos, supuestos, decisiones y riesgos pendientes.
4. No contiene credenciales ni datos personales.

Entrega 2. Plataforma base y CMS funcional

PROPÓSITO Demostrar una primera plataforma tradicional operable, segura y comprobable de extremo a extremo.

Alcance técnico obligatorio

1. VMs Ubuntu Server instaladas, actualizadas y documentadas; hostname, red estática y sincronización horaria.
2. Administración SSH segura con usuarios nominativos, sudo limitado, firewall inicial y evidencia de logs.
3. Almacenamiento, montaje persistente, propietarios, grupos, permisos y capacidad verificados.
4. DNS autoritativo o equivalente del laboratorio; no se permite sustituirlo con el archivo hosts.
5. Servidor web tradicional, virtual host, HTTP/HTTPS y certificado validados.
6. Base de datos separada y CMS funcional con al menos un flujo empresarial completo.
7. Backup previo a cambios críticos, validación de sintaxis y procedimiento de reversión.

Producto que se entrega

1. Repositorio actualizado con configuraciones saneadas, scripts, README y bitácora.
2. Informe de avance breve con topología implementada, pruebas, hallazgos y deuda técnica.
3. Demostración funcional en laboratorio.

Evidencias mínimas

- ☐ Inventario real con IP, hostname, rol, versión, recursos y puertos.
- ☐ Pruebas de resolución DNS, conectividad, escucha de puertos y acceso HTTPS.
- ☐ Registro de una transacción identificable: producto/pedido, taller/reserva o equivalente.
- ☐ Extractos relevantes de logs del sistema, SSH, DNS, web y base de datos.
- ☐ Bitácora de instalación con errores encontrados y correcciones.

Demostración y verificación

1. Acceder al CMS mediante FQDN desde un cliente autorizado.

2. Ejecutar el flujo empresarial y localizar su evidencia en la aplicación y la base de datos.
3. Explicar y mostrar qué accesos están permitidos y bloqueados.
4. Diagnosticar una falla breve seleccionada por el docente sin reinstalar el servicio.

Criterios de aceptación

1. El servicio funciona por nombre y HTTPS; no depende de IP escrita manualmente por el usuario.
2. La base de datos no queda publicada a redes no autorizadas.
3. Los permisos permiten operar sin recurrir a 777 ni compartir cuentas administrativas.
4. Las pruebas y evidencias pueden repetirse siguiendo la documentación.

Entrega 3. Publicación resiliente, contenedores y seguridad

PROPÓSITO Escalar la plataforma a dos backends y demostrar continuidad del servicio ante la falla de uno de ellos.

Alcance técnico obligatorio

1. Reproducción del CMS como aplicación multicontenedor con Docker Compose después de la línea base tradicional.
2. Dos backends funcionales y equivalentes, identificables en las respuestas o logs.
3. HAProxy como punto de entrada HTTP/HTTPS, redirección segura y health checks pertinentes.
4. Persistencia coherente de base de datos, archivos, imágenes y sesiones cuando aplique.
5. Firewall, forwarding/NAT y segmentación de flujos según la arquitectura aprobada.
6. Gestión de secretos sin incorporarlos en imágenes ni repositorios.
7. Logs del proxy y backends suficientes para demostrar distribución, fallas y recuperación.

Producto que se entrega

1. Suite reproducible con Compose, configuraciones saneadas y script de validación.
2. Informe de avance con prueba de falla, resultado, limitaciones y reversión.
3. Demostración técnica con dos backends.

Evidencias mínimas

- ☐ Archivo Compose y variables de ejemplo sin secretos.
- ☐ Configuración de HAProxy validada antes de reload.
- ☐ Secuencia de solicitudes que demuestre distribución entre backends.
- ☐ Prueba de retiro de un backend defectuoso y reintegración posterior.
- ☐ Prueba de persistencia: el mismo dato o archivo permanece disponible a través de ambos nodos.
- ☐ Matriz actualizada de flujos permitidos y bloqueados.

Demostración y verificación

1. Generar tráfico y mostrar qué backend atiende cada solicitud.

2. Detener o degradar controladamente un backend y observar el health check.
3. Verificar que el servicio continúa y que el backend sano recibe las solicitudes.
4. Recuperar el nodo, comprobar su reintegración y explicar el riesgo del proxy único.

Criterios de aceptación

1. La caída de un backend no interrumpe completamente la experiencia principal.
2. HAProxy no envía tráfico normal a un backend declarado no saludable.
3. No existe divergencia silenciosa de archivos o estado entre nodos.
4. El equipo diferencia balanceo de carga, tolerancia a falla y alta disponibilidad.

Entrega 4. Observabilidad y continuidad operacional

PROPÓSITO Pasar de una plataforma “funcionando” a una plataforma que puede ser operada, alertada y recuperada.

Alcance técnico obligatorio

1. Monitorización de proxy, backends, base de datos y host mediante Prometheus/Grafana, Zabbix o equivalente.
2. Panel con señales útiles: disponibilidad, latencia, errores, saturación, capacidad y salud de respaldos.
3. Alertas con umbral, severidad, responsable, canal y evidencia de recepción.
4. Logs consultables y correlacionables con hora consistente; retención definida.
5. Correo o relay para notificaciones o transacciones en un entorno seguro de laboratorio.
6. Backups automatizados de base de datos, archivos y configuraciones, con retención y verificación.
7. Restauración controlada en ubicación o ambiente de prueba y medición real de RPO/RTO.

Producto que se entrega

1. Dashboard, reglas de alerta y configuraciones versionadas.
2. Plan de backup, informe de restauración y registro de tiempos.
3. Runbook de operación con escalamiento y responsables.

Evidencias mínimas

- ☐ Panel con métricas antes, durante y después de una degradación.
- ☐ Alerta recibida y vinculada con la evidencia de la causa.
- ☐ Logs que permitan reconstruir una secuencia de eventos.
- ☐ Inventario de respaldos, política de retención, checksum o verificación equivalente.
- ☐ Datos identificables creados antes del backup y comprobados después de la restauración.
- ☐ Registro de RPO y RTO real, comparación con objetivos y mejora propuesta.

Demostración y verificación

1. Provocar una degradación controlada y detectar el incidente mediante la plataforma de observabilidad.
2. Explicar por qué cada métrica y umbral aporta una decisión operativa.
3. Restaurar un conjunto de datos y archivos en un destino validado.
4. Comprobar integridad funcional del CMS después de restaurar.

Criterios de aceptación

1. Una alerta crítica llega sin depender de que alguien observe permanentemente un panel.
2. Los respaldos contienen datos, archivos y configuración necesarios, no solo una carpeta vacía.
3. La restauración es demostrable y no modifica innecesariamente el ambiente productivo.
4. Las conclusiones de RPO/RTO se sustentan en marcas de tiempo y evidencias.

Entrega 5. Incidente, auditoría y defensa final

PROPÓSITO Demostrar que la plataforma integrada puede enfrentar un incidente no anunciado, recuperar el servicio y explicar las decisiones adoptadas.

Alcance técnico obligatorio

1. Plataforma completa integrada y disponible al inicio de la evaluación.
2. Auditoría final de arquitectura, superficie expuesta, permisos, secretos, capacidad, puntos únicos de falla y deuda técnica.
3. Incidente controlado asignado por el docente: DNS, TLS, proxy, backend, almacenamiento, permisos, firewall, base de datos, backup, correo o saturación.
4. Proceso de respuesta: detectar, priorizar, contener, diagnosticar, corregir, validar, comunicar y cerrar.
5. Reversión si la corrección genera efectos no deseados.
6. Defensa individual sobre arquitectura, comandos, configuración, métricas, logs, continuidad y riesgos.

Producto que se entrega

1. Informe final ejecutivo y técnico.
2. Repositorio final saneado y etiquetado con versión de entrega.
3. Runbook, inventario, matriz de pruebas, plan de continuidad y reporte de incidente.
4. Demostración y defensa final.

Evidencias mínimas

- ☐ Línea de tiempo del incidente con observaciones, hipótesis y decisiones.
- ☐ Comandos, métricas y logs relevantes antes y después de la corrección.
- ☐ Pruebas de recuperación desde cliente y servidor.
- ☐ Bitácora de cambios final y diferencias respecto de la arquitectura de Entrega 1.
- ☐ Repositorio reproducible, documentación operativa y registro de contribuciones.
- ☐ Backlog priorizado para una siguiente versión de la plataforma.

Demostración y verificación

1. Recibir el síntoma sin conocer previamente la causa raíz.
2. Registrar al menos dos hipótesis y priorizarlas mediante evidencia.
3. Recuperar el servicio sin eliminar evidencias relevantes ni aplicar soluciones inseguras permanentes.
4. Validar funcionalidad, seguridad y observabilidad tras la corrección.
5. Responder preguntas individuales y ejecutar una prueba seleccionada al azar.

Criterios de aceptación

1. La recuperación se demuestra con el flujo empresarial, no solo con el estado de un proceso.
2. La causa raíz se sostiene en evidencia y se diferencia del síntoma.
3. La corrección respeta mínimo privilegio y deja una reversión documentada.
4. Cada integrante demuestra dominio suficiente para operar la solución.
5. La documentación permite a un tercero reconstruir o mantener la plataforma.

7. Catálogo mínimo de evidencias

Tipo	Contenido esperado	Valor
Identidad	fecha/hora, equipo, ambiente y objetivo de la prueba	evita capturas ambiguas o reutilizadas
Configuración	fragmento relevante y archivo de origen saneado	muestra la decisión implementada
Estado	systemctl, docker compose, ss, ps o equivalente	confirma que el componente existe y su condición
Cliente	resolución, conexión, HTTP/HTTPS y flujo empresarial	demuestra experiencia externa
Servidor	logs, métricas, consultas o estado interno	explica qué ocurrió dentro
Falla	síntoma, hipótesis, línea de tiempo y evidencia causal	distingue diagnóstico de ensayo y error
Recuperación	pruebas posteriores y comparación antes/después	confirma que el servicio quedó estable
Reversión	backup, comando o procedimiento y validación	reduce riesgo de cambios críticos

Formato recomendado para una evidencia

1. ID y propósito: qué requisito o hipótesis se está validando.
2. Contexto: fecha/hora, host, usuario no sensible, versión y estado previo.
3. Acción: comando o procedimiento ejecutado y riesgo asociado.
4. Resultado esperado: condición concreta de éxito o falla.
5. Resultado observado: salida relevante, métrica o log.
6. Interpretación: qué demuestra y qué no demuestra.
7. Reversión o siguiente paso: cómo volver al estado seguro.

8. Incidentes posibles para la evaluación

Dominio	Ejemplos	Evidencia sugerida
Red/DNS	registro incorrecto, serial no actualizado, ruta o firewall bloqueado	dig, resolvectl, ip, traceroute, nft/ufw, logs
Web/TLS	virtual host incorrecto, certificado vencido, cadena incompleta	curl -vk, openssl s_client, logs web/proxy
Proxy/backend	health check inadecuado, backend con 500, timeout	HAProxy stats/logs, curl directo, ss, systemctl
Datos/estado	disco lleno, permisos, volumen no montado, archivo divergente	df, du, findmnt, lsblk, stat, ACL, logs

Dominio	Ejemplos	Evidencia sugerida
Base de datos	credencial, conectividad, capacidad o servicio detenido	cliente DB, ss, métricas, journal
Seguridad	intentos SSH, regla excesiva, secreto expuesto	auth logs, fail2ban, firewall, revisión repositorio
Continuidad	backup vacío, restauración parcial, retención incorrecta	inventario, checksum, prueba de restauración
Rendimiento	CPU, memoria, I/O o latencia creciente	top, free, vmstat, iostat, métricas y logs

9. Check antes de cada entrega

- ☐ La solución coincide con la topología y el inventario entregados.
- ☐ Los nombres, direcciones y segmentos están actualizados para el ambiente real.
- ☐ No existen credenciales, tokens, llaves privadas ni datos personales en el repositorio.
- ☐ Las configuraciones críticas fueron respaldadas y validadas sintácticamente.
- ☐ El equipo puede revertir el último cambio relevante.
- ☐ Las pruebas incluyen perspectiva de cliente y de servidor.
- ☐ Los logs y métricas poseen hora consistente y suficiente contexto.
- ☐ Las evidencias están identificadas y vinculadas con requisitos o hipótesis.
- ☐ El servicio fue probado después de reinicio cuando corresponde, sin reinicios innecesarios.
- ☐ Cada integrante puede explicar y operar cualquier componente esencial.

10. Entrega de archivos

8. Un archivo PDF por informe, además del formato editable cuando sea solicitado. (Documento Word)
9. Repositorio con README, inventario, arquitectura, configuraciones saneadas, scripts, pruebas, evidencias y bitácora. (github)
10. Las configuraciones deben conservar rutas y nombres claros; no entregar capturas como sustituto de archivos de configuración.
11. Identificar versión de entrega mediante etiqueta o commit y no modificarla después del cierre sin autorización.
12. Usar nombres de archivos consistentes: E##_grupo##_descripcion.ext.

13. Las fechas concretas serán publicadas en aula.usm.cl; la secuencia y ponderaciones se mantienen salvo comunicación docente formal.

11. Cierre: estándar de proyecto aprobado

Una plataforma no está terminada porque sus contenedores/servicios estén en ejecución. El proyecto se considera logrado cuando el equipo puede explicar sus decisiones, reproducir la implementación, observar su comportamiento, detectar una falla, recuperar la operación, comprobar la integridad de los datos y dejar evidencia suficiente para que otra persona continúe administrándola.

CRITERIO FINAL

Instalar es el comienzo. Operar, diagnosticar y recuperar es administración de sistemas.